Dotfiles from dotgit to bombadil

Rohit Goswami · 2021-05-02 · ramblings · tools · workflow · dots

> *Discussion on dotfile management, a meandering path to my current setup from `dotgit` to `bombadil`.*

EDIT: Superseded by my `chezmoi` configuration [described here]()

Background

No one gets very far working with stock one-size fits all tools in any discipline but it is especially true of working with computers. The right set of `dotfiles` have been compared to priming spells for invocation later, and this is probably true. More than anything else, `dotfiles` offer familiarity where there is none, be it from `cowsay` or a fancy shell prompt [^fn:1].

```
$ cowsay Welcome home $USER
  ____________________________
< Welcome home rohitgoswami >
 ----------------------------
        \   ^__^
         \  (oo)\_______
            (__)\       )\/\
                ||----w |
                ||     ||
```

Why Now?

Figure 1: Home is where the `dotfiles` are

Well more prosaically, I have recently had to partially retire my beloved ThinkPad X380 Yoga [^fn:2] for a new MacBook Pro 2019 (Intel) [^fn:3]. This is the single largest test of my `Dotfile` management system, since I now have configurations which are no longer scoped to just a Linux distribution (e.g. ArchLinux and CentOS 6), but are fundamentally not interchangeable.

Complicating matters further, `dotgit` sunset the `bash` version I have grown to love in favor of a new `python` version (explained here). I also have to manage profiles on more HPC systems than before. The time seemed ripe for a re-haul.

I'll try to justify the Figure 1 as I dissect and rebuild my `Dotfiles` as I switch from `dotgit` to `bombadil`.

Desiderata

What makes a good `dotfile` management system? Some things which are common to most/all good systems:

targets

These are configurations which are scoped to either machines or operating systems; e.g. `archlinux`, `colemak` etc.

profiles

The concept of a profile is essentially a set of targets used together; e.g. `mylaptop`, `hzhpc`

symlinks

Most management systems use symlinks to modularly swap configurations in and out; `ln -sf ....`

secrets

Commonly implemented (with varying levels of help) in the form of a `gpg` encrypted file/files

All of these features are exemplefied by the fantastic `dotgit` and no doubt its python iteration is just as brilliant. However, I am wary of using `python` for my `dotfile` management, since I tend to use transient `virtualenvs` a lot and detest having a system `python` for anything.

Over the years, I've come to also value:

simplicity

Especially true of installation proceedures, I simply need to get started quickly too often

template expansion

A rare feature to need, but one I've been addicted to since `lazybones`, `pandoc` and even `orgmode` brought a million snippets

On a probably technically unrelated note, I have recently been splurging on the "modern" (prettier) `rust` versions of standard command line tools; so I normally have `cargo` everywhere.

Starting out with Bombadil

Since toml-bombadil is:

- Written in Rust
  - installs with `cargo` as a single binary
- Supports encryption
- Supports profiles
- Has template expansion

It was the obvious choice.

```
# Get Cargo
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env
# Get toml-bombadil
cargo install toml-bombadil
```

For the rest of this post I'll assume everyone is working with a set of `dots` like my own.

```
# Get my set
export mydots="$HOME/Git/Github/Dotfiles"
mkdir -p $mydots
git clone git@github.com/HaoZeke/Dotfiles $mydots
cd $mydots
git checkout bombadil
bombadil install -c "$mydots/bombadil.toml"
```

Now we can start defining profiles in our `toml` file and link them.

```
# Get colemak and mac profiles
bombail link -p macos colemac
```

This isn't really meant to be a tutorial about `bombadil-toml`, but it might include some pointers. The remainder of the post will focus on the layout and logic of my own usage.

Shells

Shells form the core of most computing environments, and typically we would like to have basic support for at least `bash` and one additional shell. The secondary shell (`zsh` for me) is the most preferred, but often might not be avaliable [^fn:4].

Current Logic

Since many shells are somewhat compatible with each other (especially within the POSIX family); my current setup looked a bit like:

```
. ~/.shellrc # With standard agnostic commands
```

Which in turn loaded a bunch of conditionally symlinked files from profiles.

```
# Platform
#############

if [ -f ~/.shellPlatform ]; then
        . ~/.shellPlatform
fi

# Specifics
#############

if [ -f ~/.shellSpecifics ]; then
        . ~/.shellSpecifics
fi

# Wayland
#############

if [ -f ~/.waylandEnv ]; then
        . ~/.waylandEnv
fi

# XKB
######

if [ -f ~/.xkbEnv ]; then
        . ~/.xkbEnv
fi

# Nix
######

if [ -f ~/.nixEnv ]; then
        . ~/.nixEnv
fi
```

Current Approach

The problem with the older approach is that it isn't always clear where different divisions should be drawn and it isn't really flexible enough to add things arbitrarily. Basically, here there were only a few entry points. A more rational method is to emulate the work of `init.d` [^fn:5] scripts; where a set of files in a directory are all loaded if they exist [^fn:6].

This allows the previous setup to be instead refactored into the following folder setup (under `$HOME/.config/shellrc`):

- `.login.d` which has machine specific files
  - `Spack` and `Lmod` modulefiles on various HPC nodes
  - Also just things which normally run only once per login (like system diagnostics)
  - MacOS considers every terminal to be a login shell for some odd reason
- `.shell.d` which contains POSIX compliant snippets
  - This is by far the longest section
- `.bash.d` for snippets which need `bashisms`
  - Array operations
- `.zsh.d` for snippets which are specific to `zsh`
  - Plugin management

This then flows very nicely into a smaller set of core `rc` files for scripts.

```
# .bashrc / .zshrc
## Bashism (only for .bashrc)
if [[ $- != *i* ]]; then
        # shell is non-interactive. Do nothing and return
        return
fi
## Zshism (only for .zshrc)
if [[ -o interactive ]]; then
        # non-interactive, return
        return
fi

export shellHome=$HOME/.config/shellrc

# Load all files from the shell.d directory
if [ -d $shellHome/shell.d ]; then
        for file in $shellHome/shell.d/*.sh; do
                source $file
        done
fi

# Load all files from the bashrc.d directory
if [ -d $shellHome/bash.d ]; then
        for file in $shellHome/bash.d/*.bash; do
                source $file
        done
fi
```

Similarly, we can define our `zshrc`. For profiles (`.zlogin` and `.bash_profile`), we source the `rc` files along with the `login.d` scripts.

Practicalities

This method isn't really very exciting at the offset. Each target has a series of scripts which are loaded in order.

```
tree .
.
├── bash
│   └── bashrc
├── posix
│   ├── 00_warnings.sh
│   ├── 01_alias_def.sh
│   ├── 02_func_def.sh
│   ├── 03_exports.sh
│   ├── 04_sources.sh
│   ├── 05_paths.sh
│   └── 06_prog_conf.sh
├── tmux
└── zsh

4 directories, 8 files
```

This is correspondingly linked via the following snippet in `bombadil.toml`.

```
# Shells #
# POSIX
posix_warn = { source = "common/shell/posix/00_warnings.sh", target = ".config/shellrc/shell.d/00_warnings.sh"
}
posix_alias = { source = "common/shell/posix/01_alias_def.sh", target =
".config/shellrc/shell.d/01_alias_def.sh" }
posix_func = { source = "common/shell/posix/02_func_def.sh", target = ".config/shellrc/shell.d/02_func_def.sh"
}
posix_exports = { source = "common/shell/posix/03_exports.sh", target =
".config/shellrc/shell.d/03_exports.sh" }
posix_sources = { source = "common/shell/posix/04_sources.sh", target =
".config/shellrc/shell.d/04_sources.sh" }
posix_paths = { source = "common/shell/posix/05_paths.sh", target = ".config/shellrc/shell.d/05_paths.sh" }
posix_prog_conf = { source = "common/shell/posix/06_prog_conf.sh", target =
".config/shellrc/shell.d/06_prog_conf.sh" }
# Bash
bashrc =  { source = "common/shell/bash/bashrc", target = ".bashrc" }
```

The same concept (folder structure) is used for specific machines as well (e.g. `archlinux`).

```
[profiles.archlinux.dots]
archlinux_warn = { source = "archlinux/shell/posix/07_00_warnings.sh", target =
".config/shellrc/shell.d/07_00_warnings.sh" }
archlinux_alias = { source = "archlinux/shell/posix/07_01_alias_def.sh", target =
".config/shellrc/shell.d/07_01_alias_def.sh" }
archlinux_func = { source = "archlinux/shell/posix/07_02_func_def.sh", target =
".config/shellrc/shell.d/07_02_func_def.sh" }
archlinux_exports = { source = "archlinux/shell/posix/07_03_exports.sh", target =
".config/shellrc/shell.d/07_03_exports.sh" }
archlinux_sources = { source = "archlinux/shell/posix/07_04_sources.sh", target =
".config/shellrc/shell.d/07_04_sources.sh" }
archlinux_paths = { source = "archlinux/shell/posix/07_05_paths.sh", target =
".config/shellrc/shell.d/07_05_paths.sh" }
archlinux_prog_conf = { source = "archlinux/shell/posix/07_06_prog_conf.sh", target =
".config/shellrc/shell.d/07_06_prog_conf.sh" }
```

Note the way in which the files are saved to ensure the correct loading order. For `zsh` the list is a little different:

```
zshrc = { source = "common/shell/zshrc.zsh", target = ".zshrc" }
zshenv = { source = "common/shell/zsh/zshenv.zsh", target = ".zshenv" }
zsh_keys = { source = "common/shell/zsh/01_keys.zsh", target = ".config/shellrc/zsh.d/01_keys.zsh" }
zsh_func = { source = "common/shell/zsh/02_func_def.zsh", target = ".config/shellrc/zsh.d/02_func_def.zsh" }
zsh_plugins = { source = "common/shell/zsh/03_plugins.zsh", target = ".config/shellrc/zsh.d/03_plugins.zsh" }
zsh_sources = { source = "common/shell/zsh/04_sources.zsh", target = ".config/shellrc/zsh.d/04_sources.zsh" }
zsh_theme = { source = "common/shell/zsh/04a_theme.zsh", target = ".config/shellrc/zsh.d/04a_theme.zsh" }
```

Text Editors

A long time ago I switched from VIM and Sublime to Emacs. I still retain in my `dotfiles` enough syntactical sugar and targets to make `vim` and Sublime easier to use; mostly using vim-plug. Emacs has a rather complicated set of configurations which are independently managed via `doom-emacs` and have their own self documenting site.

Conclusions

Though the `shell.d` setup is still overly verbose and not as flexible as I had initially hoped, this is still a few steps ahead of my previous setup. No part of this focused on the configurations themselves (more interesting examples, of switching to Colemak are here) Will be fleshing this out more with auxilliary posts on my configuration (browsers, etc.) and its management perhaps.
